Nool

Technical Design Document

GitHub: https://github.com/guru963/isocodelabs
Live app: https://isocodelabs.vercel.app
Run, deploy and demo logins: `deployment.md` in the repository root
Date: 9 November 2026

Nool, the Tamil word for thread (நூல்), is a delay tracker for Company A, a knitwear exporter in Tiruppur with three own units and subcontracted printing and washing. Supervisors post each day's output from their phones, in Tamil, even with no signal. A risk engine compares that pace with each order's time-and-action (TNA) plan and warns a merchandiser while there is still time to catch up. Buyers in the UK and EU follow their own orders through a private read-only link, and see a delay only after the merchandiser has confirmed its reason.

1. What was built 1
2. Architecture 2
3. Data model 3
4. Main flows 3
5. How the AI works 5
6. UI and design decisions 5
7. Security and privacy 9
8. Testing and delivery 9
9. Assumptions 9
10. What I cut, and what I would do next 9

1. What was built

Every item in the core build ships end to end, in the deployed app and in the local setup. Each one is listed below with where it lives.

Brief requirement	How Nool meets it
Order setup with TNA milestones	New order screen. The plan is worked back from the ex-factory date by a template, moving off Sundays and holidays. Printing and washing can go to an outside unit. The form warns when the ship date is too close for the template's lead time.
Supervisor daily update	A Tamil phone screen, one card per stage. Counts, typed notes, photos and voice notes queue on the phone with no signal and send themselves later, never twice.
Order board with status colours	Most urgent first, with status counts that work as filters, plus buyer, unit and ship-week filters. Each row shows its pace as a thread.
Delay warning (the hard part)	A run-rate projection per stage and per order, explained in words on every screen, plus in-app notifications with mocked WhatsApp messages. A late stage also gets a catch-up plan: three worked-out ways to recover it.
AI reason category from Tamil notes	Voice notes are written down by Whisper. Each note is then translated and categorised by an LLM, and a merchandiser reviews it with the original beside the translation.
Read-only buyer link	A secret link per buyer that shows only that buyer's orders, field by field. Warnings appear only after a merchandiser confirms the reason.
Tamil interface	The supervisor screens are in Tamil by default, with an English toggle. The font draws Tamil and Latin to match.
History of every date change	Each change needs a reason and is kept in an audit log, shown on the order page and the buyer's page.
Mock payments, messages and outside systems	WhatsApp and SMS are mocked: messages are written to a log table. In-app notifications are real.
Weekly buyer export (optional)	An Excel workbook per buyer for the current week: a summary row per order, every stage's plan, and the week's date changes. The buyer downloads it from their page; the merchandiser from Buyer links.

Optional depth that was built: both optional items in the brief, board filters beyond status (buyer, unit, ship week) and the weekly buyer export, plus a catch-up planner for late stages, an ex-factory runway timeline, scheduled reminders and escalations, late entry of a missed day's count, voice notes, and a CI/CD pipeline that deploys only after the tests pass.

2. Architecture

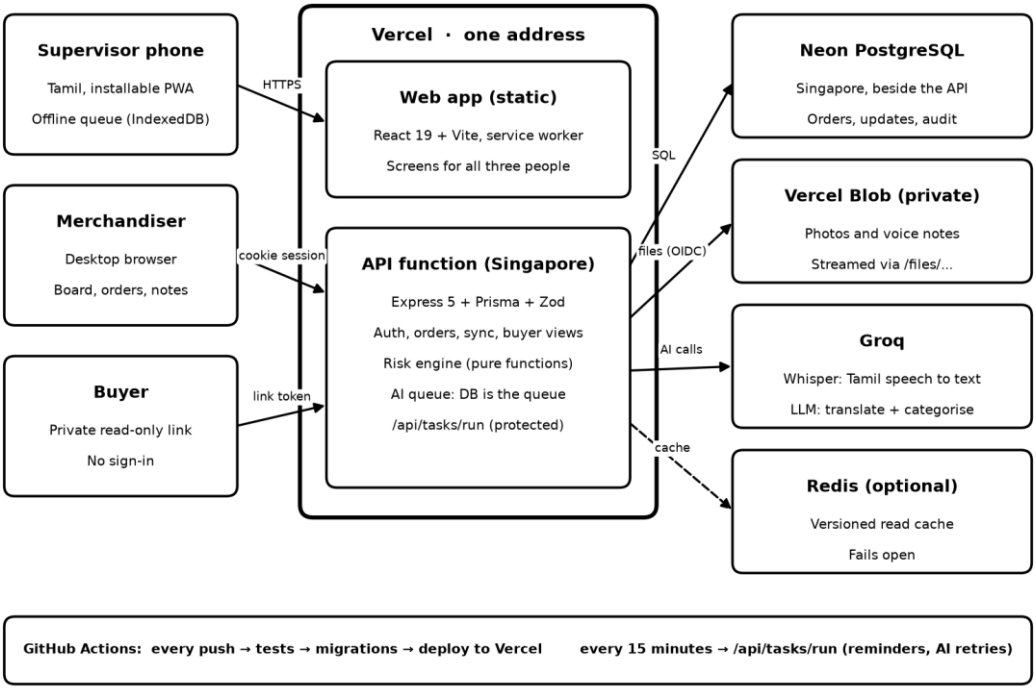


Figure 1. Three people, one address on Vercel, and the services behind it.

Nool is one TypeScript monorepo with two apps. `apps/web` is a React app that the supervisor installs on the phone. `apps/api` is an Express API on PostgreSQL. In production both run on Vercel at one address: the screens are static files, and the whole API is a single Vercel Function in Singapore, next to the Neon database. GitHub Actions tests every push, applies migrations, deploys, and wakes the app every 15 minutes for its scheduled work.

2.1 Components

Layer	Technology	Why
Web	React 19, Vite 8, React Router 7, vite-plugin-pwa	One codebase for three people. Installs on a cheap Android phone and works offline. Screens are split by who uses them, so a supervisor's phone never downloads the merchandiser screens.
API	Node 22, Express 5, Zod, Prisma 6	Small and well understood. Zod checks every request body; Prisma gives typed queries and migrations.
Database	PostgreSQL 17 (Neon in production, Docker locally)	The data is relational: orders, milestones, daily updates, notes. Syncing a supervisor's update needs a transaction with a row lock.
AI	Groq: `openai/gpt-oss-120b` and `whisper-large-v3`	Fast, with a free tier and one key for both. The LLM supports strict JSON output, and Whisper handles Tamil.
Files	Vercel Blob, private (local disk in development)	A function's disk is wiped between requests. Files stay private and are streamed back through the app.
Cache	Redis, optional	Repeat reads skip the database. Every write invalidates them at once, and the app works with no Redis.
Hosting and delivery	Vercel, GitHub Actions	One address, a region next to the database, no server that sleeps, and nothing deployed without passing tests.

2.2 Key decisions

- **One API, organised by domain.** Auth, orders, updates, issues, buyer views, notifications and the risk engine are separate modules in one app. At one factory's volume, separate services would add network calls and deployments without solving a real problem.
- **Risk is calculated when read, never stored.** It depends on today's date. A stored status would stay green after supervisors stop reporting. The engine is a set of pure functions with no database access, so it is tested on its own.
- **The database is the AI queue.** A note waits as a row marked pending until it succeeds or runs out of attempts. It survives restarts and needs no broker. BullMQ on Redis was considered and rejected: a worker process for a few notes a day is extra infrastructure, and a serverless function cannot host one.
- **Offline first for supervisors.** Each update gets a random ID on the phone, so a retried send never counts twice. A newer count for the same stage and day replaces the older one, and an update that arrives after a newer one is ignored.
- **One address for everything.** Screens, API and files share one origin. The session is a first-party, httpOnly cookie, there is no CORS to configure, and buyer links point at the app's own domain.
- **Serverless, with the timers moved out.** On a server, timers read notes and raise reminders. On Vercel, AI work finishes after the reply under `waitUntil`, and a protected `/api/tasks/run` endpoint does the rest. GitHub Actions

calls it every 15 minutes, and Vercel Cron calls it twice a day as a backup. Render was rejected because its free tier sleeps.

- **Region matched to the database.** The function first ran in Vercel's default US region while Neon is in Singapore, and a health check took 0.62 s. Moving it to Singapore brought that to about 0.25 s, most of which is the network.
- **Speech-to-text as a service, not a model to host.** A self-hosted Whisper on Hugging Face was built first, but its PyTorch install failed on the host. Groq's Whisper uses the same key as the classifier and needs nothing to run.

3. Data model

Fifteen tables in PostgreSQL, managed by Prisma migrations. The ones the flows below depend on:

Table	Key fields	Notes
Unit	name, type (own or subcontract)	Unit 1 to 3, plus the printing and washing partners.
User	role, language, unit	Merchandiser or supervisor. A supervisor belongs to one unit and sees every open stage there.
Buyer, BuyerLink	name, country; tokenHash, expiresAt, revokedAt	Only a SHA-256 hash of the link's token is stored.
TnaTemplate, TnaTemplateStep	stage, sequence, daysBeforeExFactory	Plain knit and Printed knit. A plan is worked back from the ex-factory date.
Order, OrderSize	poNumber (unique), buyer, style, colour, totalQty, exFactoryDate, unit, merchandiser	Size split kept per order.
Milestone	stage, plannedDate, plannedQty, doneQty, completedAt, unit	Inspection and ex-factory are events with no quantity. They are marked passed or shipped.
DailyUpdate	id (from the phone), workDate, qty, note, photo, voice, capturedAt, enteredOn, supersededAt	One live count per stage per day. Older ones are kept, marked superseded.
Issue	originalText, translatedText, aiStatus, aiAttempts, aiError, aiCategory, aiConfidence, finalCategory, status	A note and its AI result. `finalCategory` is set on review.
AuditLog	entity, field, old and new value, reason, user, time	Every date change, with its reason.
Holiday	date, unit (or all)	Plans and pace skip them.
Notification, MessageLog	kind, dedupeKey (unique), read time; channel, recipient	The unique key makes each reminder happen once.

Rules the model keeps: risk status is not a column. A milestone's `doneQty` is updated in the same transaction as the update that changes it. A shipped order is one whose ex-factory milestone is complete. A buyer link's token exists only in the link itself.

4. Main flows

4.1 Setting up an order

1. The merchandiser enters the PO, buyer, style, colour and pieces per size, then chooses a unit, a template and an ex-factory date.
2. The API works each stage date back from the ex-factory date by the template's day offsets. A date that lands on a Sunday or holiday moves earlier, because moving it later would eat into the next stage.
3. It also returns the earliest ex-factory date that leaves every stage on or after today. A closer date is allowed, but the form says which stage would start behind and offers the earliest date in one click.
4. Printing and washing can be given to an outside unit. On save, the order, its sizes and its milestones are written in one transaction.

4.2 The supervisor's daily update

1. The phone keeps the last worklist from the server, so the screen opens with no signal. Each card is a stage under way in the supervisor's unit.
2. A count, an optional note, a compressed photo (longest side 1280 px) and a voice note are saved on the phone in IndexedDB, under a new random ID.
3. When online, files upload first and the counts follow, in batches of up to 50. A file the server cannot store waits on its own; the rest of the queue still goes.
4. For each update, the server locks the milestone row and checks that the stage belongs to the supervisor's unit and that the total stays within the planned quantity. It then answers per update: accepted, duplicate (the same ID again), stale (an older capture than the one stored) or rejected with a reason.
5. A count missed yesterday is asked for the next morning, and can be added for the last working day only. Its `enteredOn` date shows the merchandiser it was late.

4.3 The delay projection

This is the core of the app. For each stage still in progress, on each read:

1. **Pace:** the average of the last three reported days, adding together reports from more than one supervisor on the same day.
2. **Needed:** the pieces left, divided by the working days left until the stage's milestone (Monday to Saturday, minus holidays).

3. **Projection:** the pieces left at the current pace, counted forward in working days, gives the date the stage will finish. Its difference from the milestone is the slip.
4. **Order:** each stage's slip pushes the stages after it, which keep their planned length. The latest result is the projected ex-factory date. A missed ex-factory date stays red until the order ships.

Status	When
Delayed (red)	Slip of 3 or more days, the milestone has passed with pieces left, nothing made across the reported days, or the ex-factory date has passed
At risk (amber)	Slip of 1 or 2 days, or no report for 3 or more working days
On track (green)	Projected to finish on or before the milestone
No reports	Production is due but nothing has been reported. It ranks above green, so silence never looks healthy
Not started, done	Before the previous stage's milestone; all pieces made

For example, B-PO-88240's finishing stage makes 800 pieces a day against the 934 it needs, so it projects to 11 November against a 10 November milestone: 1 day late, and at risk. Every screen says this in words, such as "Heading for 11 Nov, 1 day late", so colour is never the only signal.

Catch-up planner

A warning alone leaves the merchandiser to work out what to do. For every late stage the API also returns a catch-up plan, worked out by pure functions from the same figures as the warning, with three options:

- **Make more each day:** the daily output that meets the milestone from tomorrow, against today's pace. For B-PO-88240: 934 a day instead of 800, +17%. An increase beyond 25%, more than a line can usually stretch to, is shown but not offered as a plan.
- **Work days normally off:** the pieces left at today's pace need more working days than remain, so the planner lists the earliest Sundays and holidays before the milestone that close the gap, or says how many days would still be missing. For B-PO-88240: work Sunday 8 November.
- **Move the deadline:** let the stage end on its projected date and take the time from the next milestone, but only while the next stage keeps at least one working day. For B-PO-88240 this is refused, because inspection is the very next day. When it is possible, one button opens the date form with the new date and a reason already filled in, so the change still goes through the audit log.

When no single option works, the planner tries every lever at once: output stretched by 25% and as many days off as it takes. If that still ends late, it pushes the later stages by their planned lengths, as the projection does, to give the earliest realistic ex-factory date. For B-PO-88213, 200 a day against 1,180 needed, that is 1 January 2027 instead of 20 November. One button opens the ex-factory form with that date filled in, and another drafts a message asking the buyer to agree it (section 5.3). A stage that is on time, has no reported output, or has stopped entirely gets no plan, because there is nothing reliable to plan from. The planner has its own unit tests on the sample figures.

4.4 The buyer's link

- A link carries 32 random bytes after the #, so the secret is never sent in a request line or written to server logs. The page sends it as a header, and the API looks up its SHA-256 hash. A wrong, expired or revoked link gets the same error.
- Every buyer query goes through a repository built for one buyer ID, so another buyer's order cannot be reached by changing an ID. The buyer's view lists its fields one by one, so a new column can never leak by accident. Internal figures, notes, units and staff stay out.
- A warning appears only when a merchandiser has confirmed a note on a stage that is still open, and the order is at risk or delayed. A photo waits while its note is unreviewed, in case it shows the problem.
- **The weekly export** is an Excel workbook built from the same buyer-scoped repository and the same buyer view as the page, so it can never show more than the link does. It covers Monday to Sunday of the current week, in three sheets: a summary row per order (status, confirmed reason, the stage now and its progress, what comes next), every stage's plan with the ones due this week marked, and the date changes made this week with their reasons. A buyer downloads it with the link's token; a merchandiser downloads any buyer's copy from Buyer links. Excel was chosen over PDF because buyers paste progress into their own trackers, and it needs no fonts on the server.

4.5 Date changes, reminders and notifications

A date change needs a reason of at least three characters, and is written to the audit log in the same transaction. Reminders are checked every 5 minutes on a server, or every 15 minutes through the tasks endpoint on Vercel:

Notification	Who	When	Channels
Missed count	Supervisor	After 17:30 India time, a stage under way has no count today	In-app, WhatsApp (mock)
Missed yesterday	Supervisor	After 10:00, the last working day's count is still missing	In-app, WhatsApp (mock)
Escalation	Merchandiser	After 10:00, the same count is still missing	In-app, WhatsApp (mock)
Note waiting	Merchandiser	A note unreviewed for 2 hours	In-app
Order delayed	Merchandiser	An order turns red, once per ex-factory date	In-app, WhatsApp (mock)

5. How the AI works

Supervisors write short, informal notes, usually in Tamil and sometimes mixed with English, or they speak them. The AI does two jobs, and a person always has the last word.

5.1 The pipeline

1. **Saved first.** The update commits with its note. The note gets an issue row marked pending, and the AI is started after the commit without being waited on, so a slow or failing model never holds up a floor update.
2. **Speech to text, for voice notes.** The recording is read back from storage and sent to Whisper (`whisper-large-v3`) with the language set to Tamil, which is faster and stops a short clip being heard as another language. The transcript is saved as the note's original text before anything else, so a later failure never repeats it.
3. **Translate and categorise.** The text goes to `openai/gpt-oss-120b` on Groq with a strict JSON schema: an English translation, one of eight categories (power cut, machine breakdown, worker absence, fabric delay, subcontractor delay, quality rework, trims shortage, other) and a confidence from 0 to 1. The prompt describes the factory and each category, and says the note is data to translate, not instructions, as a guard against prompt injection. The note is wrapped in tags.
4. **Checked, not trusted.** The reply is parsed and validated with Zod. An invalid reply counts as a failure and is retried. An unknown category keeps the translation but becomes "other" at zero confidence, so a person looks at it.
5. **Reviewed.** The merchandiser sees the original beside the translation, can edit both the translation and the category, and confirms or dismisses the note. A result under 70% confidence is marked for a closer look. Only a confirmed note can reach a buyer.

5.2 When the AI fails

Situation	What happens
No API key configured	Every note goes straight to a person, marked as not read by the AI
Timeout, outage, or an invalid reply	Retried up to 3 attempts, by the queue's sweep, then left for a person. A merchandiser can send it back with "Ask the AI again"
Recording with no speech	Sent to a person to listen to; the classifier is not called
Voice file missing	Sent to a person, without spending attempts
Low confidence (under 0.7)	Marked on the review screen as needing a closer look
Server restarted mid-call	The note is still pending in the database, so the next sweep picks it up

5.3 Drafting the message to the buyer

When a stage cannot be recovered, the merchandiser has to ask the buyer for a new date. The AI writes the draft, but does none of the reasoning: the planner works out the date and the actions, and passes only those facts (PO, stage, the confirmed reason if any, what the factory is doing, the agreed and new dates) to the same Groq model. The reply is checked: any number in it that was not in the facts, such as a date or quantity the model made up, rejects the whole draft, and so does a draft that leaves out the PO or the new date. A rejected draft, a failed call or a missing key all fall back to a fixed template from the same facts. The draft is editable, says whether the AI or the template wrote it, and is never sent from the app.

Measuring it: the review screen shows how often merchandisers keep the AI's category, from `finalCategory` against `aiCategory`. On the sample notes it kept 2 of 2. **A known limit:** Whisper can invent a word from silence or machine noise, which is one more reason a merchandiser listens before confirming a voice note.

6. UI and design decisions

Three people use Nool in three very different places, so each gets a different screen, held together by one visual language.

- **Light only.** Supervisors read the screen under factory tube lights and merchandisers in a bright office, so there is no dark theme.
- **Colour from the client's own world.** Peacock blue and rani pink are the classic pairing on Kanchipuram silk, set on greige, the unbleached cotton the factory starts from. Turmeric marks thread. Red, amber and green mean order status only, and always come with a word.
- **One typeface, Anek Tamil,** from an Indian foundry, whose Tamil and Latin letters are drawn to match. Only the weight axis ships, at about 93 KB, for low-memory phones. Inputs are 16 px so a phone never zooms.
- **Thread as the signature.** Progress is drawn as thread: solid for work done, stitches for work left. It appears in the pace bar, the seam of stages, the runway and the stitched labels around PO numbers.
- **Familiar controls.** Layout and controls follow common patterns; the character is in colour, type and the thread. Every dropdown and calendar is drawn in the same style instead of the browser's own.

6.1 Order board

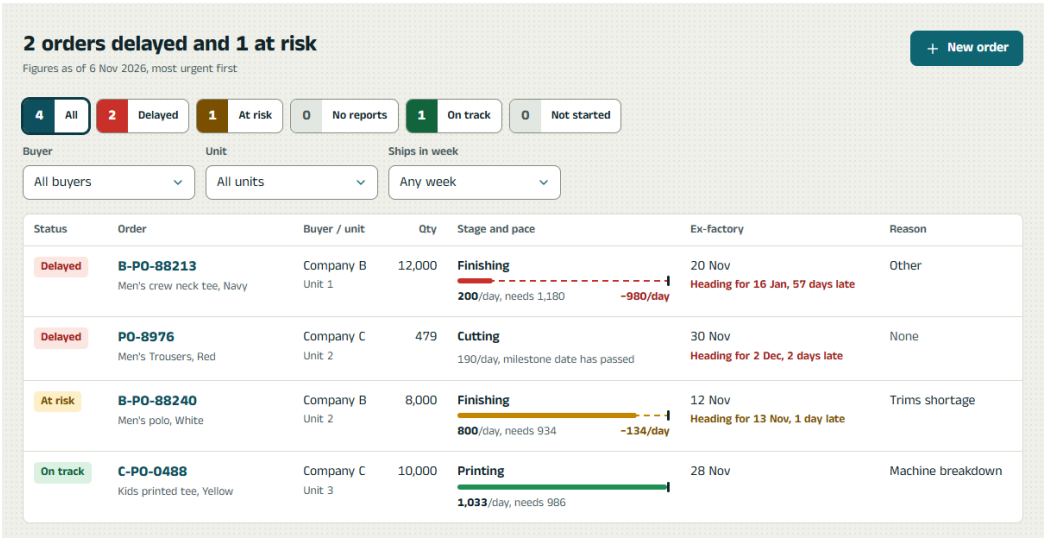


Figure 2. The merchandiser's order board.

The title states the position in words, such as "2 orders delayed and 1 at risk", and rows are sorted most urgent first. The status counts look like a dye shade card and double as filters. Each row's pace bar shows the pieces a day being made against the pieces needed, with the shortfall drawn as stitches, so the size of a problem is visible before reading a number.

6.2 Order page

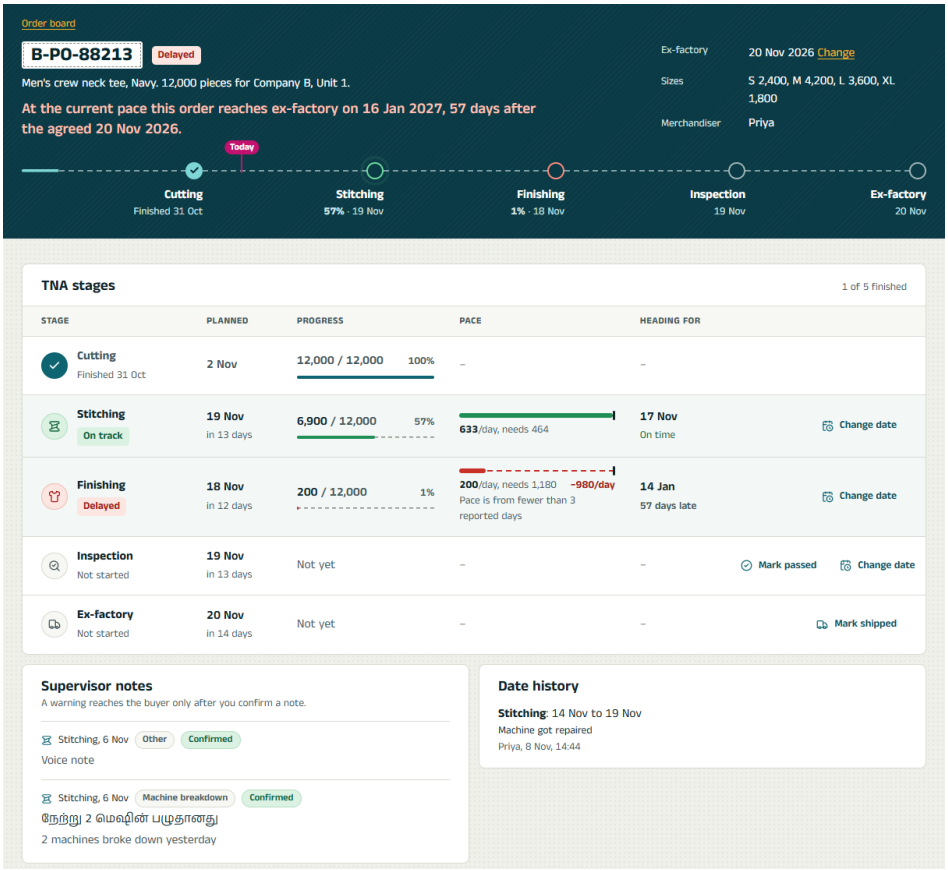


Figure 3. One order: the summary in words, the seam of stages, and the TNA table.

The header says in one sentence what the colour means, such as "At the current pace this order reaches ex-factory on 16 Jan 2027, 57 days after the agreed 20 Nov". Below it, the seam shows every stage and today's position. The stage table gives each stage's progress, pace and projected finish, with quiet row actions for changing a date or marking a stage passed. Notes, date history and paged daily reports follow.

A late stage gets a "How to catch up" action that opens its plan under the row. Options that recover the stage on their own have a solid border; ones that cannot have a dashed one and say why.

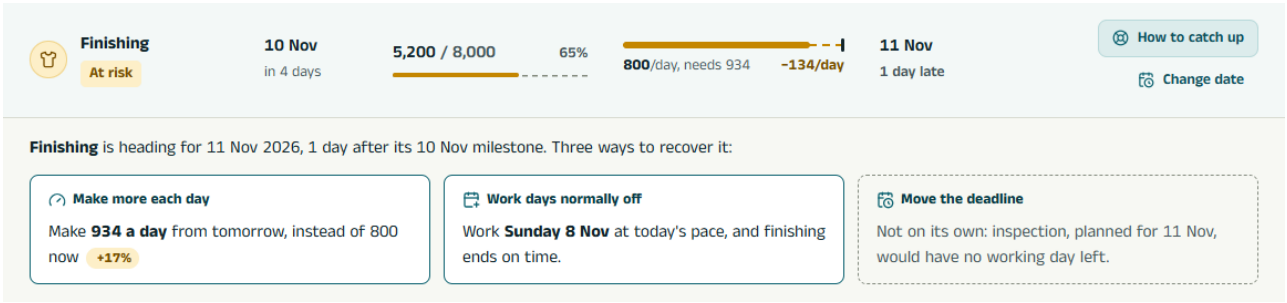


Figure 4. The catch-up plan under a late stage.

6.3 New order

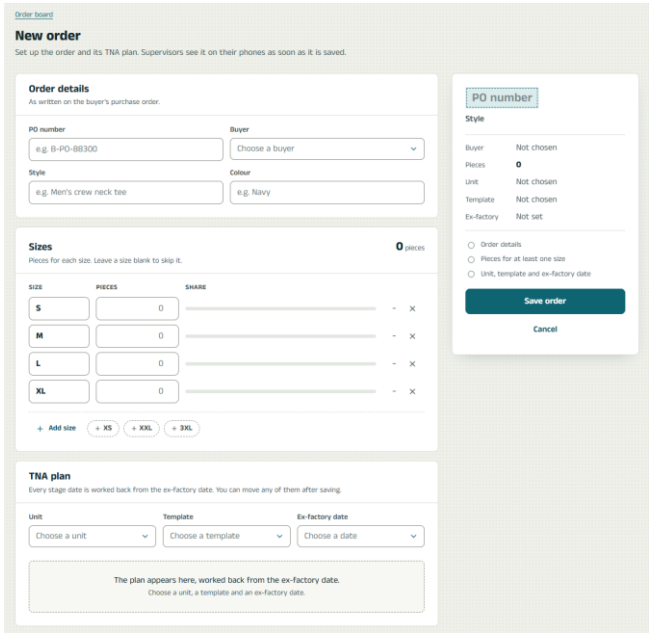


Figure 5. The form beside a live summary.

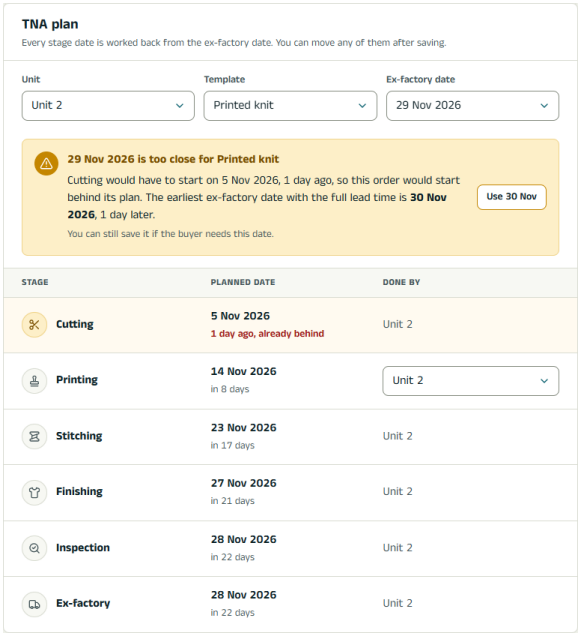


Figure 6. A ship date too close for the template's lead time.

The form sits beside a summary that fills in as it is typed: the PO on a stitched label, the size split, the plan's stage count, the ship date, and a checklist of what is still missing. The plan preview appears as soon as a unit, template and date are chosen. A date too close for the template's lead time gets one plain warning with the way out.

6.4 The supervisor's phone



Figure 7. Today's work, in Tamil.

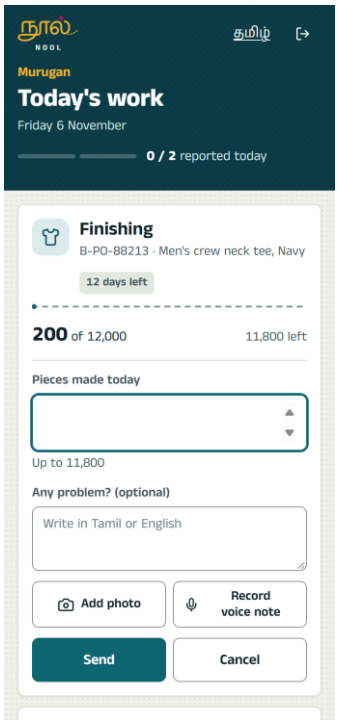


Figure 8. Entering a count.

One column of cards, one per stage, with buttons at least 44 px tall for one hand on a cheap phone. Each card shows how far the stage has come and how many days are left. The count has one large field, and the note can be typed or spoken. A pill at the top says whether everything has been sent or is waiting for signal, so a supervisor never wonders if a count was lost.

6.5 The buyer's page

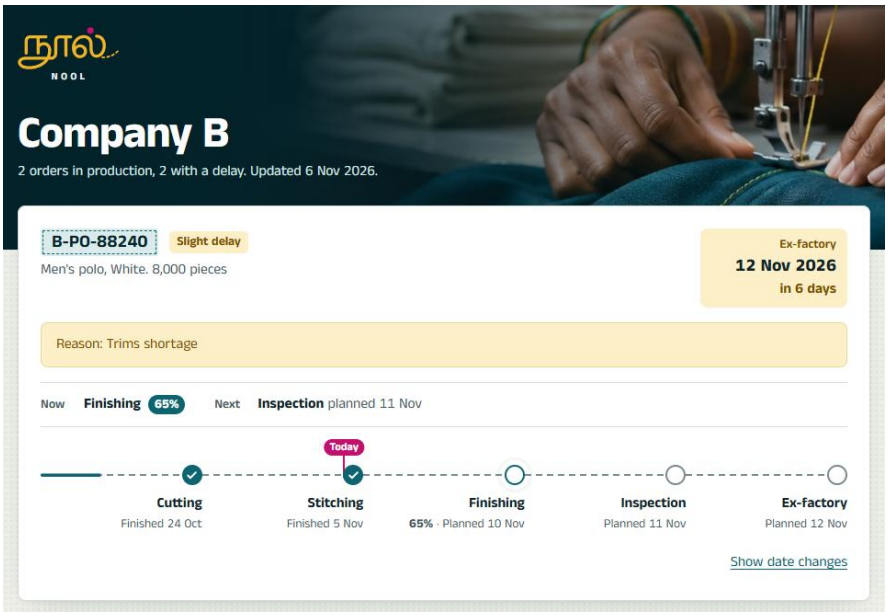


Figure 9. What a buyer sees: ship date first, then what is being made now and what comes next.

A calm page a buyer can forward. Each order answers three questions in order: when it ships, what is being made now, and what comes next. Status stays neutral unless a merchandiser has confirmed a delay, and then the confirmed reason is shown. There are no internal figures, notes or names.

A light outline button on the photo band downloads the week's report. It is there when wanted, but never competes with the orders below it.

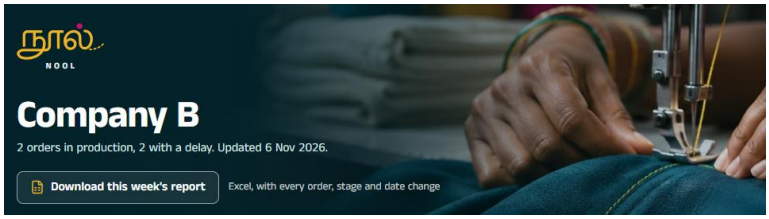


Figure 10. The report button on the buyer's page.

Company B: weekly order progress

Week of 2 Nov 2026 to 8 Nov 2026. Figures as of 6 Nov 2026. Read-only, from the factory floor's daily reports.

PO	Style	Colour	Pieces	Ex-factory	Status	Reason	Now	Done	Next	Planned
B-PO-88240	Men's polo	White	8,000	12 Nov 2026	In progress		Finishing	65%	Inspection	11 Nov 2026
B-PO-88213	Men's crew neck tee	Navy	12,000	24 Nov 2026	In progress		Stitching	56%	Finishing	18 Nov 2026

Figure 11. The weekly report's summary sheet, as Excel prints it.

6.6 Buyer links

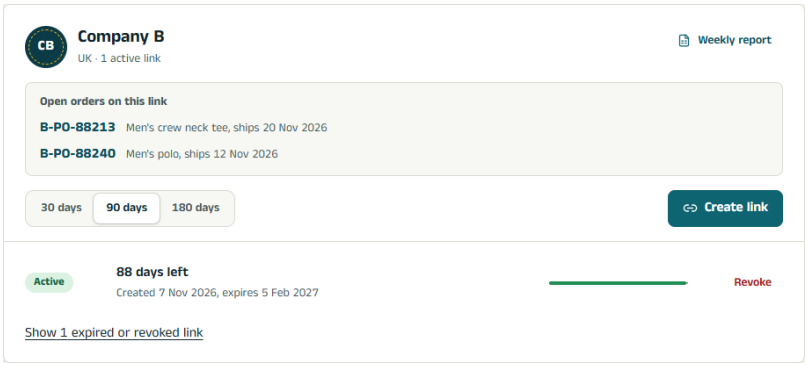


Figure 12. A buyer's card: the weekly report, the orders the link shows, then the links.

Each buyer's card lists the open orders its link will show, so the merchandiser knows exactly what is being shared. The weekly report sits quietly in the card's corner. A new link appears once, as a stitched ticket with Copy and Preview. Each link line shows its state and how much of its life is left.

7. Security and privacy

Concern	How it is handled
One buyer seeing another's orders	Buyer queries go through a repository fixed to one buyer ID, and fields are listed one by one. Integration tests try to reach another buyer's order and expect a refusal
Sessions	Signed JWT in an httpOnly, SameSite=Lax cookie, Secure in production. 12 hours for merchandisers; 30 days for supervisors, whose phones may stay offline
Sign-in attempts	Rate-limited to 20 per 15 minutes
Roles	Every route checks the role, and a supervisor can only report stages of their own unit
Buyer link secrets	32 random bytes, stored only as a SHA-256 hash, kept out of URLs sent to the server, with an expiry and revocation
Uploaded files	The file type is decided from its first bytes, not its name. Files are private in Blob and only reachable through the app
Scheduled endpoint	Needs a secret bearer token, compared in constant time
Secrets	None in the repository. Settings come from the environment, and deployment.md lists names only
Headers	Helmet's defaults, with no referrer sent

8. Testing and delivery

- **108 API unit tests:** the risk engine, the catch-up planner and the buyer-message check, the calendar, TNA planning, AI reply parsing, notification rules, the cache, and the scheduled and file endpoints.
- **22 web unit tests:** the offline queue, against an in-memory IndexedDB, including duplicate sends, a lost connection, a missed day, and a file the server cannot store.
- **86 integration tests:** the whole API against a real PostgreSQL, reseeded for each file. They cover sign-in and roles, order setup, sync and its edge cases, buyer isolation, notes and the AI with fake models, voice notes, notifications, missed days, the cache, and the weekly report, whose test opens the workbook and checks it holds no other buyer's orders, units, people or note text.
- **CI/CD:** every push type-checks, runs the unit tests and builds; the integration tests run against a fresh PostgreSQL 17 with every migration applied from empty. On `main`, a deploy job then applies migrations to Neon, deploys to Vercel and checks the live health endpoint. The commit history shows the build in order.

9. Assumptions

Only the sample orders come from the brief: PO numbers, buyers, styles, quantities, ex-factory dates, current stages and quantities done. Everything else below is an assumption, kept in one place in the code so the client can change it.

Assumption	Value	Where
Working week	Monday to Saturday, minus listed holidays	risk/calendar.ts
Pace window	Average of the last 3 reported days	risk/engine.ts
At risk from	1 day of slip	risk/engine.ts
Delayed from	3 days of slip	risk/engine.ts
Stale data	No report for 3 working days	risk/engine.ts
Later stages	Keep their planned length when an earlier stage slips	risk/engine.ts
Catch-up	A line can stretch its daily output by at most 25%; a day worked off runs at today's pace; a moved deadline must leave the next stage at least 1 working day	risk/catchUp.ts
Templates	Plain knit: cutting 18 days before ex-factory, stitching 6, finishing 2, inspection 1. Printed knit adds printing at 14, with cutting at 24	seed/sampleData.ts
Sample detail	Size splits, units, people, daily figures and notes; daily figures add up to the brief's quantities done	seed/sampleData.ts
Demo date	Today is pinned to 6 Nov 2026 so the sample dates stay meaningful	APP_TODAY
Review threshold	AI confidence under 0.7 is marked for a closer look	ai/classification.ts
Reminder times	17:30 for supervisors, 10:00 for the missed day and escalation (India time)	notifications
Note waiting	2 hours before the merchandiser is reminded	notifications/rules.ts
Late updates	An offline update is accepted up to 30 days late; a missed count can be added for the last working day only	updates
Buyer links	Valid 30, 90 or 180 days	BuyerLinks screen
Tamil wording	Interface and message wording should be checked by a native speaker, as should the logo's letterforms	i18n, messages

10. What I cut, and what I would do next

10.1 Cut, on purpose

- **Real WhatsApp and SMS.** Mocked as the brief allows: messages are written to a log with their channel and recipient, ready for a provider.

- **Admin screens** for users, units, templates and holidays. They come from the seed; the tables exist, so screens can be added without schema changes.
- **Editing an order after setup,** other than its dates. Sizes and quantities are fixed once saved.
- **A supervisor for the washing partner.** The sample data has none, so a stage sent there cannot be reported yet.

10.2 Next

1. Connect a WhatsApp Business provider to the existing message log, starting with the delayed-order alert to merchandisers.
2. Send the weekly report by itself every Monday morning, through the scheduled job, to a contact saved for each buyer.
3. Add admin screens for templates, holidays and people, so the factory can tune the plan without a developer.
4. Guard against Whisper inventing words from silence, using its no-speech probability, and have a Tamil speaker check transcripts and wording on real recordings.
5. Track each merchandiser's AI agreement over time, and adjust the prompt or threshold where they disagree most.
6. Move the Blob store to Singapore beside the function, and add error tracking and alerts for the scheduled job.
7. Add browser end-to-end tests for the supervisor's offline flow on a real low-end Android phone.